

AURA

UI/UX Design & Build Documentation

MVP Scope: Mail · Calendar · Time

From design system to shipped iOS app — a complete build guide

Version 1.0 · September 2026

Table of Contents

- 1. Scope & How to Use This Document
- 2. Brand & Design Language
- 3. Design Tokens: Color, Type, Spacing, Elevation
- 4. Component Library
- 5. Information Architecture
- 6. Onboarding Flow — Screen by Screen
- 7. The Today Screen — Full Specification
- 8. The Inbox & Reply Screen — Full Specification
- 9. Settings Screen
- 10. Notifications, Widgets & Live Activities
- 11. Empty, Loading & Error States
- 12. Motion & Micro-interaction Guidelines
- 13. Accessibility Requirements
- 14. Technical Architecture
- 15. Data Model
- 16. API Design
- 17. Integration Guide: Mail (Gmail & Microsoft Graph)
- 18. Integration Guide: Calendar (EventKit)
- 19. Integration Guide: Maps & Wake-Time Calculation
- 20. The AI Layer: Prompts, Guardrails & Evaluation
- 21. Security & Privacy Implementation
- 22. Engineering Plan: Team, Sprints, Milestones
- 23. QA & Testing Strategy
- 24. Launch Checklist

1. Scope & How to Use This Document

This document is the detailed design-and-build companion to the Aura MVP Lean Sketch. It assumes the three-module MVP scope (Mail Manager, Calendar Manager, Time Manager, plus a shared To-Do layer) and goes one level deeper: an actual design system, full screen specifications, and a concrete engineering path from empty repository to a shipped iOS app in the Swiss App Store.

It is written so that a designer can build a working Figma file directly from Sections 2–13, and an engineering team can start sprint planning directly from Sections 14–24, without needing another round of interpretation.

2. Brand & Design Language

2.1 Personality

Aura should feel calm, competent, and quietly confident — the visual equivalent of a very good personal assistant who never makes a fuss. Three words to test every design decision against:

- Calm — no gamification, no streaks, no red badges screaming for attention unless something genuinely needs a decision.
- Competent — information is precise and dated/timed, never vague ("7:10 AM" not "soon").
- Discreet — Aura is a layer over the user's existing tools, not a flashy new destination; visuals should feel closer to a premium utility (health, finance) than a social or entertainment app.

2.2 Naming conventions in UI copy

- Aura always speaks in first person as a quiet assistant: "I drafted a reply" rather than "A reply has been generated."
- Every AI-generated block of content is labelled, never presented as if the user wrote it — this is a trust requirement, not just a style choice.

3. Design Tokens: Color, Type, Spacing, Elevation

3.1 Color system

Swatch	Token	Hex	Usage
	Aura Violet (primary)	#5B4B8A	Primary actions, active nav state, key numbers (wake time)
	Aura Violet Light	#8A7CC0	Secondary accents, AI-suggestion card borders

Swatch	Token	Hex	Usage
	Ink (text)	#1F1B2E	Primary text
	Slate (secondary text)	#6B6B76	Secondary/meta text, timestamps
	Surface Tint	#F7F5FB	Card backgrounds, alternating rows
	Success	#3E8E5A	Confirmed/sent/approved states
	Warning	#C97A2B	Deadlines approaching, needs-review flags

Dark mode: invert to a near-black (#121016) background with the same violet accent lightened slightly (#A79AE0) for sufficient contrast; never pure black, to keep the calm/premium feel rather than an OLED-battery-saver aesthetic.

3.2 Typography

Style	Font / size / weight	Usage
Display	SF Pro Display, 34pt, Bold	The wake-up time on the Today screen — the single most important number in the app
Title	SF Pro Display, 22pt, Semibold	Screen titles, section headers
Body	SF Pro Text, 17pt, Regular	Email content, reply drafts, general reading
Caption	SF Pro Text, 13pt, Regular	Timestamps, meta info, sender names
Label	SF Pro Text, 13pt, Semibold, uppercase, letter-spaced	Section labels like "AURA SUGGESTS"

All text styles must scale with Dynamic Type; the Display style should have a defined maximum scale to avoid breaking the Today screen's hero layout on the largest accessibility sizes (cap around the "XL" Dynamic Type step, then wrap to two lines gracefully).

3.3 Spacing & layout grid

- Base unit: 4pt. Standard paddings: 8 / 12 / 16 / 24 / 32.
- Screen margins: 20pt left/right on iPhone.
- Card corner radius: 20pt (soft, calm feel); buttons: 14pt; chips/badges: full pill radius.

3.4 Elevation

- Aura avoids heavy drop shadows. Use a 1px hairline border (#E8E5F2) plus a very soft shadow (8% opacity, 12pt blur) to separate cards from background — signals "calm," not "floating dashboard."

4. Component Library

Component	Description	Key states
Aura Suggestion Card	Tinted card (Surface Tint background, violet left border accent) used any time Aura presents AI-generated content	Default / Editing / Sent-Confirmed
Time Slot Chip	Pill-shaped tappable chip showing a proposed meeting time	Default / Selected / Disabled (slot no longer free)
Deadline Badge	Small rounded badge showing relative due date ("due in 3 days")	Neutral (>3 days) / Warning (≤2 days) / Overdue (red, rare)
Wake-Time Hero	Large display-style time with a one-line reason beneath it, used only on the Today screen	Computed / Manual Override / Recalculating
Primary Button	Filled violet, 14pt radius, used for one primary action per screen (Send, Connect, Approve)	Default / Pressed / Disabled / Loading
Secondary Button	Outline violet on white/dark surface, used for Edit/Dismiss actions	Default / Pressed / Disabled
List Row (Inbox/To-Do)	Standard row: leading avatar/icon, title, meta line, trailing badge/timestamp	Unread / Read / Actioned

5. Information Architecture

The MVP uses a two-tab structure plus a modal onboarding flow. This is intentionally flatter than the eventual seven-module nav rail described in the full product documentation — it should feel like there's nothing to learn.

Tab	Purpose	Contains
Today	The one screen a user should need to check once a day	Wake-time hero, today's unified calendar strip, shared to-do list
Inbox	Where email triage and AI reply drafting happens	Message list, reading pane, Aura reply panel

A settings gear icon lives in the top-right corner of both tabs rather than occupying a third tab slot, since it is visited rarely.

6. Onboarding Flow — Screen by Screen

1. Welcome screen: one sentence value proposition ("Aura drafts your replies and tells you exactly when to wake up") with a single "Get Started" button — no feature tour carousel, which tends to be skipped anyway.

2. Connect Mail: native OAuth sheet for Gmail or Microsoft; explicit, plain-language explanation of what Aura reads and why ("Aura reads new messages to draft replies and spot deadlines — nothing is sent without your approval") shown before the OAuth prompt, not buried in T&Cs.
3. Connect Calendar: system EventKit permission prompt, preceded by the same plain-language pattern ("Aura reads your calendar to know when you're free and to compute your wake-up time").
4. Learn Your Habits: a short, skippable 3-question flow (typical commute method, typical wake buffer/prep time, preferred reply tone: formal/casual) to seed the habit model instead of waiting weeks to learn it passively.
5. First Value Screen: Aura immediately surfaces one real result from the connected accounts — either a drafted reply to an existing unanswered email, or tomorrow's computed wake-up time — so the "aha moment" happens inside onboarding, not after.

Total onboarding should be completable in under 90 seconds excluding OAuth provider load time; every permission ask is paired with a one-line reason immediately before the system prompt.

7. The Today Screen — Full Specification

7.1 Layout (top to bottom)

Region	Content	Notes
Header	Date + greeting ("Good evening")	Minimal, never a hero image
Wake-Time Hero	Large time (e.g., "7:10") + one-line reason ("First meeting 9:00 · 25 min commute")	Tapping expands a breakdown: prep time, commute time, buffer. A pencil icon allows manual override for that single day only.
Today's Calendar Strip	Horizontally scrollable chips of today's events with time and location	Events with a detected travel-time buffer show a small route icon
Needs Your Attention	Compact list combining: unanswered emails Aura flagged, to-dos due today/soon, any reply drafts awaiting approval	This is the single "queue" — deliberately merged rather than three separate lists, to protect the promise of one glanceable screen

7.2 Wake-Time Hero interaction detail

- Default state: shows tonight's computed time for tomorrow, calculated after the last calendar sync of the day (recomputed if a calendar event changes).
- Tap to expand: reveals the calculation as a short vertical list (Wake → Get ready (25 min) → Leave by 8:15 → Commute (25 min, live traffic) → Arrive 8:40 for a 9:00 start).

- Manual override: a single "Set for tomorrow only" action; it never silently disables the automatic system for future days, avoiding a confusing "why did it stop working" support issue.
- Late-notice flow: if traffic conditions materially change after the wake time was set, a push notification offers a one-tap "Let them know you'll be a few minutes late" action that opens an Aura-drafted short message to the relevant meeting contact.

8. The Inbox & Reply Screen — Full Specification

8.1 Layout (iPhone: stacked; iPad: split view)

Region	Content	Notes
Message list	Standard inbox, newest first, with an "Aura drafted a reply" indicator dot on relevant threads	Swipe actions: Archive, Mark handled
Reading pane	Full message body with inline highlights	Detected meeting-time requests and deadlines are underlined in violet with a small tappable icon
Aura Reply Panel	Labelled "Aura suggests", shows the drafted reply text, and — when relevant — 2–3 Time Slot Chips for proposed meeting times	Buttons: Send (primary), Edit (secondary, opens the draft in an editable text view), Discard (tertiary, low-emphasis text link)

8.2 Reply drafting interaction detail

6. Aura shows the draft collapsed to 3 lines by default with a "Show full draft" expand affordance, so the panel doesn't dominate the screen for short replies.
7. Time Slot Chips reflect genuinely free calendar time, already including Aura's computed travel buffers — tapping one inserts it into the draft in natural language, it does not just paste a raw timestamp.
8. Sending is always one explicit tap on a clearly labelled Send button; there is no auto-send setting in the MVP, by design (see Section 20.3, Guardrails).
9. After sending, the card animates to a quiet "Sent" confirmation state and collapses, keeping the inbox list uncluttered.

8.3 Deadline extraction interaction detail

- An underlined deadline phrase in the message body shows a small badge (e.g., "due in 3 days") on tap, with an inline "Add to To-Do" / "Already added" toggle — extraction is proposed, not silently written, though in practice most users will leave it on auto-add after the first few confirmations build trust.

9. Settings Screen

Section	Contents
Connected Accounts	Mail account(s), calendar account(s), each with a clear "Disconnect" action and last-sync timestamp
Reply Preferences	Tone (formal/casual), signature, languages Aura should draft in
Wake-Time Preferences	Default prep-time buffer, preferred earliest/latest wake bounds, commute method (car/public transit/bike/walk)
Notifications	Toggle and timing for: nightly wake-time notification, late-running alert, deadline reminders
Privacy & Data	Link to the plain-language privacy explainer, a visible activity log ("what Aura has read and drafted"), and full account/data deletion

10. Notifications, Widgets & Live Activities

Surface	Trigger	Content
Push notification	Nightly, after the day's last calendar sync	"Tomorrow: wake at 7:10 — first meeting 9:00"
Push notification	Live traffic degrades after the wake time was already set	"Traffic is heavier than expected — leave 10 min earlier, or let [contact] know you'll be late"
Push notification	Reply-worthy email sits unanswered past the configured threshold	"You haven't replied to [name] yet — I drafted something"
Lock Screen widget	Always available	Tomorrow's wake time (or today's if before it), and count of items needing attention
Live Activity / Dynamic Island	Active on the morning of a wake-time countdown	Countdown to "leave by" time, updating live with traffic

11. Empty, Loading & Error States

- Empty Inbox (no items needing attention): a calm confirmation state ("You're all caught up") rather than a blank screen — reinforces the feeling of being taken care of.
- First-sync loading: a short, honest progress state ("Reading your last few days of mail...") rather than a generic spinner, since the user is implicitly trusting Aura with sensitive access at this exact moment.

- OAuth/connection failure: a specific, non-technical explanation and a direct retry action ("We couldn't connect to Gmail — check your connection and try again"), never a raw error code.
- AI drafting failure or low-confidence output: the Aura Suggestion Card explicitly shows "I wasn't confident enough to draft this one" rather than presenting a low-quality guess as if it were reliable.

12. Motion & Micro-interaction Guidelines

- Standard transition: 250ms ease-out for card appearances (e.g., the Aura Reply Panel sliding in) — quick enough to feel responsive, slow enough to feel calm rather than jumpy.
- The Wake-Time Hero should use a subtle number-rolling animation when the time is recalculated (e.g., traffic changes), so the user visibly sees why it moved rather than it silently jumping.
- Avoid celebratory animations (confetti, bounce) for routine actions like sending a reply — reserve any delight moment for genuinely rare milestones (e.g., "You've saved roughly 3 hours this month" surfaced occasionally, not per action).

13. Accessibility Requirements

- Full VoiceOver labelling for AI-suggestion cards, explicitly announcing "AI-drafted reply, not yet sent" so screen-reader users get the same trust cues as sighted users.
- Minimum contrast ratio of 4.5:1 for all text against its background in both light and dark mode, including inside the violet-tinted suggestion cards.
- Dynamic Type support up to the largest accessibility sizes, with the Wake-Time Hero wrapping gracefully as described in Section 3.2.
- All primary actions (Send, Approve, Connect) reachable and operable via Switch Control and Voice Control, not just direct touch.

14. Technical Architecture

The MVP architecture is intentionally simple: a SwiftUI client, a slim backend for OAuth/session/orchestration, and a small number of external integrations. No banking, payment, or location-heavy services are needed at this stage.

Layer	Responsibility	Technology
iOS client	UI, EventKit read access, local notification scheduling, widget/Live Activity rendering	Swift, SwiftUI, EventKit, WidgetKit, ActivityKit, iOS 17+

Layer	Responsibility	Technology
Backend API	Auth/session, OAuth token storage & refresh, orchestration between mail/calendar/AI, push dispatch	Node.js or Python service behind an API gateway, hosted in an EU/Swiss region
Mail connectors	Gmail API and Microsoft Graph API clients; webhook or polling-based new-mail detection	Provider SDKs + a lightweight normalization layer producing one internal "Message" shape
AI orchestration	Prompt construction, calling the LLM, parsing structured output, confidence scoring	Claude via the Anthropic API
Data store	User profile, preferences, habit averages, to-dos, message metadata (not full raw bodies long-term)	PostgreSQL
Push	Nightly wake-time notification, real-time alerts	Apple Push Notification service (APNs)

15. Data Model

Core entities, deliberately minimal for the MVP:

```
User  id, email, locale, tone_preference, created_at ConnectedAccount  id, user_id,
provider (gmail | microsoft | icloud_calendar), oauth_token_ref, last_synced_at
MessageSummary  id, user_id, account_id, subject, sender, received_at,
detected_deadline (nullable), detected_meeting_request (bool), reply_status (none |
drafted | sent | discarded) CalendarEvent (cached view, source of truth stays in
EventKit/provider)  id, user_id, title, start_at, end_at, location (nullable),
source_calendar TodoItem  id, user_id, title, due_at (nullable), source (manual |
mail_extraction), source_message_id (nullable), status (open | done) WakeTimePlan
id, user_id, for_date, computed_wake_at, reason_breakdown (json), manual_override
(bool)
```

Full raw email bodies are fetched just-in-time from the provider API for display and AI processing, and are not persisted beyond a short cache needed for the reply-drafting session — only the derived MessageSummary is stored long-term, in line with the minimum-necessary-retention principle from the privacy section.

16. API Design

A small set of backend endpoints is sufficient for the MVP; everything else (raw mail/calendar reads) happens through direct, short-lived, per-request calls to the provider APIs orchestrated by the backend.

Endpoint	Method	Purpose
/auth/oauth/callback	POST	Exchange provider OAuth code for tokens, create/link ConnectedAccount

Endpoint	Method	Purpose
/sync/mail	POST (internal/cron or webhook-triggered)	Pull new messages, run AI extraction, create MessageSummary + TodoItem records, push reply drafts
/messages/{id}/reply	GET	Fetch the current AI-drafted reply and time-slot options for a message
/messages/{id}/reply	POST	Send the (possibly edited) reply via the provider's send API; mark reply_status = sent
/todos	GET / POST / PATCH	List, create, and update to-do items
/waketime/today	GET	Fetch the current computed wake-time plan and its reason breakdown
/waketime/override	POST	Set a one-day manual override
/settings	GET / PATCH	Read/update user preferences (tone, buffers, notification timing)

17. Integration Guide: Mail (Gmail & Microsoft Graph)

10. Register OAuth applications with Google Cloud Console (Gmail API, scopes limited to gmail.readonly and gmail.send — never the broader full-mailbox-modify scope) and Microsoft Entra (Mail.Read and Mail.Send only).
11. Implement the OAuth authorization-code flow inside the iOS app using ASWebAuthenticationSession, sending the resulting code to the backend to exchange for tokens server-side (tokens are never handled directly by the client).
12. Use provider push mechanisms where available (Gmail's Pub/Sub watch, Microsoft Graph change notifications) instead of constant polling, to keep the integration efficient and closer to real-time.
13. Normalize incoming messages into the internal MessageSummary shape before anything touches the AI layer, so the reply-drafting and deadline-extraction logic is provider-agnostic.
14. Sending: use the provider's own send endpoint (Gmail's messages.send, Graph's sendMail) so replies appear in the user's Sent folder exactly as if typed by hand.

18. Integration Guide: Calendar (EventKit)

15. Request EventKit full-access calendar permission on-device (no backend calendar OAuth needed for Apple/iCloud calendars; Google/Exchange calendars already synced into iOS Calendar are covered transparently through EventKit).
16. Read events for the relevant look-ahead window (next 14 days is sufficient for meeting-slot suggestions and wake-time calculation) and cache a lightweight CalendarEvent view server-

side only as needed to support AI reasoning — treat EventKit as the source of truth, not the cache.

17. For users whose primary work calendar is Exchange/Outlook rather than a calendar synced into iOS, offer the Microsoft Graph Calendar API as a secondary connector using the same OAuth app already registered for Mail.
18. Free/busy slot detection for reply suggestions: compute open windows by subtracting all events (across every connected calendar) from the user's working-hours preference, then apply the travel-time buffer logic from Section 19 before presenting Time Slot Chips.

19. Integration Guide: Maps & Wake-Time Calculation

19. Use Apple's MapKit / MKDirections API for commute-time estimation between the user's home location (set once during onboarding or inferred from overnight location patterns) and the first event's location.
20. Wake-time formula: $\text{wake_at} = \text{event_start} - \text{travel_time (live, from MapKit)} - \text{prep_time (from user-set or learned average)} - \text{a small fixed buffer (default 5 min)}$.
21. Habit learning for prep_time: maintain a rolling average of the gap the user has historically needed between a manually-set alarm and their first event, adjusting gradually rather than reacting to a single outlier morning.
22. Recompute the plan whenever a relevant calendar event changes or nightly traffic patterns for the known route shift meaningfully, and only surface a new push notification if the change is significant (e.g., more than 10 minutes) to avoid notification fatigue.

20. The AI Layer: Prompts, Guardrails & Evaluation

20.1 Reply drafting task

Input: the message thread, the sender's relationship history with the user (prior tone, frequency), and a list of genuinely free calendar slots already computed by the backend (Section 18). Output: a drafted reply in the user's preferred tone, plus 0–3 slot references if the message is scheduling-related.

- The model should never be asked to determine calendar availability itself — that is computed deterministically by the backend and passed in as ground truth, keeping the LLM's job to language generation rather than fact computation.
- Confidence scoring: if the model's own assessment of the message's intent is ambiguous (e.g., unclear whether a reply is even expected), the app shows the message without a suggestion rather than forcing a low-quality draft.

20.2 Deadline/commitment extraction task

Input: message body. Output: a structured object (title, due_at, confidence) or null if no genuine commitment is detected. This should run as a small, cheap, high-volume classification+extraction call, separate from the more expensive reply-drafting call, so cost scales sensibly with mail volume.

20.3 Guardrails (non-negotiable for MVP)

- No automatic sending, ever, in v1 — every reply requires an explicit tap, regardless of confidence score.
- No drafting replies to messages that look like sensitive categories (legal, medical, financial disputes) without an explicit "this one needs your full attention" flag instead of a draft, to avoid a wrong-tone AI reply in a high-stakes context.
- Every draft is generated fresh per session rather than cached indefinitely, so edits to the underlying thread are always reflected.

20.4 Evaluation before launch

- Build a labelled test set of real (anonymized/synthetic) email threads covering scheduling requests, deadline mentions, and ambiguous cases, and measure draft-acceptance-without-edit rate and deadline-extraction precision/recall before opening the beta.
- Track the acceptance metrics from Section 7 of the Lean Sketch (draft acceptance rate, wake-time reliance) as the primary live signal once real users are on the product.

21. Security & Privacy Implementation

- OAuth tokens encrypted at rest and scoped to the minimum permissions described in Section 17 — never request full-mailbox-modify or full-account-access scopes.
- Raw message bodies are processed in-memory for the duration of a request and not written to long-term storage; only the derived MessageSummary and any user-approved drafts persist.
- All traffic over TLS; backend hosted in an EU/Swiss region to align with the nFADP-based data-residency posture described in the main product documentation.
- A visible in-app activity log lists every message Aura has read a summary of and every draft it has produced, satisfying the transparency commitment even at MVP scope.
- Full account deletion removes ConnectedAccount tokens immediately and purges all derived data (MessageSummary, TodoItem, WakeTimePlan) within a defined short window (e.g., 30 days, documented in the privacy policy).

22. Engineering Plan: Team, Sprints, Milestones

22.1 Minimum team

- 1 iOS engineer (SwiftUI), 1 backend engineer, 1 designer (part-time acceptable after the design system in Sections 2–4 is locked), 1 founder/PM.

22.2 Sprint plan (2-week sprints, ~16 weeks total)

Sprints	Focus
1–2	Design system in Figma (Sections 2–4); backend skeleton, auth, OAuth app registrations.
3–4	Mail connector (Gmail first), message normalization, basic Inbox UI (no AI yet).
5–6	EventKit integration, free/busy computation, Today screen skeleton with a static (non-AI) wake-time placeholder.
7–8	AI layer: reply drafting + deadline extraction, wired into Inbox UI with the Aura Suggestion Card component.
9–10	Wake-Time Hero fully wired to MapKit + habit learning; notifications and widget.
11–12	Microsoft Graph mail connector (second provider), Settings screen, activity log, account deletion flow.
13–14	Onboarding flow polish, empty/error states, accessibility pass, internal dogfooding.
15–16	Closed beta (Zurich/Basel cohort from the Go-to-Market plan), bug fixing, evaluation metrics wired up.

23. QA & Testing Strategy

- Unit tests for the wake-time calculation logic (Section 19) given fixed inputs — this is a pure function and should be the most rigorously tested piece of logic in the app, since its correctness is the core trust promise.
- AI evaluation harness (Section 20.4) run against every prompt change before it ships, to catch regressions in draft quality or extraction accuracy.
- Manual QA checklist per OAuth provider (Gmail, Microsoft) covering token refresh/expiry and revoked-access handling, since broken auth silently breaks the whole product experience.
- Accessibility audit (VoiceOver pass, Dynamic Type at largest size, contrast check) before each beta milestone, not only before public launch.

24. Launch Checklist

- Privacy policy and in-app data-use explainer reviewed against nFADP requirements; Swiss legal entity in place if payments/App Store payout require it.

- OAuth apps for Gmail and Microsoft verified/approved by the respective platforms (both require a verification review before scopes like mail-reading go live for the public — this has real lead time and should be started early).
- Crash reporting and basic product analytics wired up, hosted in an EU/Swiss-friendly provider, with no sensitive message content ever sent to analytics.
- Closed beta metrics (Section 7 of the Lean Sketch) reviewed against the go/no-go bar before public App Store release.
- App Store listing localized in German (launch language), with French and Italian queued for the fast-follow localization pass.
- Support channel in place (even a simple shared inbox) before public launch, given the trust-sensitive nature of the product — early users will have real questions about what Aura can see.